

Capítulo 3

Geração de Números Aleatórios

Edmilson Marmo Moreira

Universidade Federal de Itajubá - UNIFEI

Instituto de Engenharia de Sistemas e Tecnologia da Informação - IESTI

“Só há um caminho para a felicidade. Não nos preocuparmos com coisas que ultrapassam o poder da nossa vontade”.

Epicuro

1 Considerações Iniciais

A geração de variáveis aleatórias é essencial em diversas áreas do conhecimento, como simulação computacional, estatística, criptografia e teoria das filas. Em muitas dessas aplicações, é necessário representar fenômenos incertos, como tempos de espera em sistemas de atendimento, falhas em equipamentos ou oscilações em mercados financeiros.

Para modelar esses fenômenos, utilizam-se variáveis aleatórias que seguem distribuições probabilísticas específicas. No entanto, como os computadores são dispositivos determinísticos, eles não produzem números verdadeiramente aleatórios. Em vez disso, utilizam algoritmos capazes de gerar sequências de números **pseudoaleatórios**, isto é, sequências determinísticas que simulam o comportamento aleatório com base em regras matemáticas bem definidas.

Dada a importância da geração de números pseudoaleatórios para simulações e outras aplicações científicas e tecnológicas, é fundamental compreender os métodos disponíveis, suas propriedades e formas de validação. Este material apresenta os conceitos fundamentais, incluindo métodos baseados em congruência e em sequências de Fibonacci, bem como técnicas para gerar variáveis aleatórias a partir de números uniformemente distribuídos no intervalo $[0, 1)$, permitindo a obtenção de distribuições discretas e contínuas.

2 Números aleatórios

A geração de números aleatórios por computador baseia-se em uma rotina que, a partir de uma **semente** inicial, produz duas sequências associadas:

1. **Sequência de números inteiros:** valores gerados em um intervalo discreto pré-definido (ex: $\{0, 1, \dots, \alpha - 1\}$), que, idealmente, seguem uma distribuição uniforme.

2. **Sequência de números reais:** valores no intervalo contínuo $[0, 1)$, obtidos pela normalização dos inteiros

$$r_i = \frac{s_i}{\alpha},$$

que podem ser transformados para gerar outras distribuições (ex: normal, exponencial).

Se o gerador for de boa qualidade, a sequência de inteiros $\{s_i\}$ apresenta uma distribuição aproximadamente uniforme no domínio discreto, e os valores normalizados $\{r_i\}$ aproximam uma distribuição uniforme contínua no intervalo $[0, 1)$. No entanto, como o processo é determinístico e o conjunto de valores possíveis é finito, os números gerados são tecnicamente pseudoaleatórios, e a qualidade da uniformidade depende do algoritmo utilizado.

Cada gerador utiliza uma função de atualização determinística, que mapeia um inteiro s_i em outro s_{i+1} dentro do mesmo domínio. A geração é iterativa:

1. Parte-se de uma semente s_0 ;
2. Gera-se a sequência de inteiros $\{s_i\}$ por meio da função de atualização;
3. Obtém-se a sequência de reais $\{r_i\}$ conforme:

$$r_i = \frac{s_i}{\alpha}$$

Como o conjunto $\{0, 1, \dots, \alpha - 1\}$ é finito e a função de atualização é determinística, toda sequência gerada é eventualmente **cíclica**, ou seja, os valores começarão a se repetir após certo número de iterações. O número de elementos distintos gerados antes do início da repetição é denominado **período** da sequência.

A qualidade de um gerador depende principalmente do comprimento de seu período e da uniformidade na distribuição dos valores ao longo do tempo. Geradores de boa qualidade possuem longos períodos e produzem sequências com boa dispersão dos valores no domínio.

Diversos métodos podem ser utilizados para a geração de números pseudoaleatórios. Entre os mais conhecidos estão:

- o **método do meio-quadrado**;
- os **métodos congruentes**;
- os **métodos de partição de palavra**;
- o **método de Fibonacci**.

2.1 Método do meio-quadrado

O **método do meio-quadrado**, proposto por John von Neumann, segue os seguintes passos:

1. Escolhe-se um número inteiro inicial (semente).
2. Eleva-se esse número ao quadrado.
3. Extraem-se os dígitos centrais do resultado, que formam o próximo número da sequência.
4. Repete-se o processo iterativamente.

Apesar de sua simplicidade, o método apresenta limitações importantes. Algumas sementes conduzem rapidamente a sequências constantes ou a ciclos muito curtos, comprometendo a qualidade da pseudoaleatoriedade.

Por exemplo, considere a semente $s_0 = 5731$. Aplicando o método com extração de 4 dígitos centrais, obtém-se:

i	n_i	n_i^2	n_{i+1}
0	5731	32844361	8443
1	8443	71284249	2842
2	2842	08076964	0769
3	0769	00591361	5913
4	5913	34963569	9635
5	9635	92833225	8332
6	8332	69422224	4222
7	4222	17825284	8252
8	8252	68095504	0955
9	0955	00912025	9120
$\vdots$	$\vdots$	$\vdots$	$\vdots$

Dessa forma, a sequência gerada seria:

$$5731 \rightarrow 8443 \rightarrow 2842 \rightarrow 0769 \rightarrow 5913 \rightarrow \dots$$

Embora historicamente relevante, esse método caiu em desuso devido à sua instabilidade e à curta duração dos ciclos gerados. Algumas variações, como o aumento do número de dígitos ou o uso de deslocamentos, podem mitigar essas limitações, mas não são suficientes para torná-lo adequado a aplicações mais exigentes.¹

¹Este método é apresentado principalmente com fins didáticos e históricos, não sendo recomendado para aplicações práticas de simulação.

O código Python a seguir contém uma implementação do método do meio-quadrado.

```
def meio_quadrado_tabela(semente, n_digitos=4, iteracoes=10):
    resultados = [semente]

    for _ in range(iteracoes):
        quadrado = str(resultados[-1] ** 2).zfill(2 * n_digitos)
        total_digitos = len(quadrado)
        meio = total_digitos // 2
        inicio = max(0, meio - n_digitos // 2)
        fim = inicio + n_digitos
        trecho = quadrado[inicio:fim]
        # Ajustar caso o trecho seja menor que n_digitos
        trecho = trecho.zfill(n_digitos)
        resultados.append(int(trecho))

    return resultados

# Exemplo de uso
sequencia = meio_quadrado_tabela(5731, n_digitos=4, iteracoes=10)
for i, num in enumerate(sequencia):
    print(f"{i}: {num}")
```

O algoritmo inicia com uma **semente** numérica de tamanho fixo e, a cada iteração, eleva esse número ao quadrado, extrai os dígitos centrais do resultado (ajustando com zeros à esquerda, se necessário) e utiliza esse valor como o próximo número da sequência. Esse processo é repetido por várias iterações, gerando uma sequência pseudoaleatória por meio de manipulação simples de inteiros e strings.

2.2 Métodos de congruência

Entre os diversos métodos existentes para a geração de números pseudoaleatórios, destaca-se o método da **congruência multiplicativa**, amplamente utilizado por sua simplicidade e eficiência.

Nesse método, a sequência de inteiros pseudoaleatórios é gerada recursivamente segundo a seguinte fórmula de atualização:

$$s_i = (\beta \cdot s_{i-1}) \bmod \alpha$$

onde:

- s_0 é a **semente** inicial;
- β é um multiplicador (número inteiro positivo);
- α é o **módulo**, que define o intervalo dos valores gerados $\{0, 1, \dots, \alpha - 1\}$.

A tabela a seguir ilustra um exemplo da aplicação do método, considerando a semente $s_0 = 7$, com os parâmetros $\alpha = 16$ e $\beta = 5$:

i	s_{i-1}	$\beta \cdot s_{i-1}$	s_i	$r_i = \frac{s_i}{\alpha}$
1	7	35	3	0,1875
2	3	15	15	0,9375
3	15	75	11	0,6875
4	11	55	7	0,4375
5	7	...	...	...

Observe que $s_4 = s_0$, o que indica que a sequência entrou em um ciclo de período 4. A escolha dos parâmetros influencia diretamente o comportamento do gerador, afetando tanto o comprimento do ciclo quanto a distribuição dos valores. Por exemplo, ao se utilizar sementes pares com esse mesmo conjunto de parâmetros, obtêm-se, em geral, períodos ainda menores.

A escolha adequada dos parâmetros α e β é essencial para garantir a qualidade dos números gerados. Um critério básico é evitar que β seja múltiplo de α , pois isso pode fazer a sequência convergir rapidamente para o valor zero ou entrar em ciclos curtos.

Mais genericamente, os métodos de congruência podem ser estendidos com fórmulas da forma:

$$s_i = (\delta s_{i-1}^2 + \gamma s_{i-1} + \beta) \bmod \alpha$$

com $\alpha, \beta, \gamma, \delta$ sendo constantes inteiras. Essa generalização dá origem às seguintes classificações:

- **Congruência multiplicativa:** $s_i = (\beta s_{i-1}) \bmod \alpha$
- **Congruência aditiva:** $s_i = (s_{i-1} + \beta) \bmod \alpha$
- **Congruência mista:** $s_i = (a s_{i-1} + c) \bmod m$
- **Congruência quadrática:** envolve o termo δs_{i-1}^2

O gerador de congruência mista, baseado na fórmula linear $s_i = (a \cdot s_{i-1} + c) \bmod m$, foi amplamente utilizado em bibliotecas clássicas de geração de números pseudoaleatórios, como a função `rand()` da linguagem C. Embora métodos mais sofisticados sejam atualmente preferidos, sua simplicidade ainda justifica o uso em aplicações específicas.

A seguir, apresenta-se uma implementação em Python do método de congruência multiplicativa, que gera uma sequência de números pseudoaleatórios com base em uma semente inicial e nos parâmetros α e β , conforme ilustrado no exemplo anterior:

```
def gerador_congruente_multiplicativo(semente, beta, alpha, iteracoes):
    print(f"{'i':<3} {'s[i-1]':<7} {'B·s[i-1]':<10} {'s[i]':<5} {'r[i]':<6}")
    print("-" * 35)
    s = semente
    for i in range(1, iteracoes + 1):
        produto = beta * s
        s_novo = produto % alpha
        r = s_novo / alpha
        print(f"{'i':<3} {'s':<7} {'produto':<10} {'s_novo':<5} {'r':<6.4f}")
        s = s_novo

# Parâmetros conforme o exemplo
semente = 7
beta = 5
alpha = 16
iteracoes = 10

gerador_congruente_multiplicativo(semente, beta, alpha, iteracoes)
```

2.3 Métodos de partição de palavra

Os métodos de partição de palavra exploram a representação binária de números inteiros, utilizando operações como deslocamentos, rotações e combinações por operações lógicas (*bitwise*). São amplamente empregados em implementações de baixo nível devido à sua eficiência computacional.

A seguir, apresenta-se um método ilustrativo que combina partição assimétrica, operações de mistura e rotação. A palavra binária de k *bits* é dividida em duas partes — **alta** e **baixa** — utilizando deslocamento e máscara, respectivamente. Em seguida, essas partes são combinadas por meio da operação XOR, somadas a uma constante ímpar e rotacionadas, com uma nova operação XOR, promovendo a difusão dos *bits*.

Sejam:

- ρ o parâmetro de rotação;
- b o número de *bits* utilizados na parte baixa;
- $m = 2^b - 1$ a máscara correspondente.

O método é definido pelas seguintes etapas:

1. Parte Alta:

$$\text{alta} = s_{i-1} \gg \rho$$

2. Parte Baixa:

$$\text{baixa} = s_{i-1} \& m$$

3. Combinação:

$$\text{temp} = (\text{alta} \oplus \text{baixa}) + c, \quad (c \text{ ímpar})$$

4. Rotação e mistura:

$$s_i = \text{rot_left}(\text{temp}, \rho) \oplus s_{i-1}$$

A operação `rot_left` representa uma rotação circular à esquerda dos *bits*, preservando o tamanho da palavra.

A tabela a seguir ilustra o processo, usando palavras de 4 *bits*, com partição em 2 *bits* na parte baixa ($b = 2$), rotação $\rho = 1$, $s_0 = 13$ (1101), $m = 3$ (0011) e $c = 5$ (0101):

i	s_{i-1}	$\text{alta} \oplus \text{baixa}$	temp	s_i	r_i
1	1101	$0110 \oplus 0001 = 0111$	1100	0100	0.250
2	0100	$0010 \oplus 0000 = 0010$	0111	1010	0.625
3	1010	$0101 \oplus 0010 = 0111$	1100	0011	0.188
4	0011	$0001 \oplus 0011 = 0010$	0111	1101	0.813
5	1101	$0110 \oplus 0001 = 0111$	1100	0100	0.250
6	0100	$0010 \oplus 0000 = 0010$	0111	1010	0.625
7	1010	$0101 \oplus 0010 = 0111$	1100	0011	0.188

Note que a sequência entra rapidamente em ciclo, o que é esperado devido ao número finito de *bits* utilizados. No exemplo considerado, o ciclo possui período 4:

$$\underbrace{13}_{i=0} \rightarrow 4 \rightarrow 10 \rightarrow 3 \rightarrow \underbrace{13}_{i=4} \rightarrow 4 \rightarrow \dots$$

A seguir, é apresentado o código Python do método descrito.

```
def gerador_por_particao(k, s0, c, b=2, rho=1, max_iter=20):
    resultados = []
    historico = {s0: 0}

    m = (1 << b) - 1
    s = s0

    for i in range(1, max_iter + 1):
        alta = s >> rho
        baixa = s & m
        temp = (alta ^ baixa) + c

        temp_rot = ((temp << rho) | (temp >> (k - rho))) & ((1 << k) -
1)
```

```

s_novo = temp_rot ^ s
r_i = s_novo / (2**k)

resultados.append((format(s_novo, f"0{k}b"), r_i))

if s_novo in historico:
    return resultados, historico[s_novo]

historico[s_novo] = i
s = s_novo

return resultados, -1

# Teste com os seguintes parâmetros
resultados, ciclo_inicio = gerador_por_particao(4, 13, c=5, b=2, rho=1,
max_iter=10)

print("Iter | Estado (bin) | r_i")
print("-" * 30)
for i, (s_bin, r_i) in enumerate(resultados, 1):
    print(f"{i:4d} | {s_bin:>12} | {r_i:.3f}")

```

O programa implementa o método descrito para geração de números pseudoaleatórios por partição assimétrica de palavra. A cada iteração, a palavra atual é dividida em duas partes — alta e baixa —, utilizando deslocamento e máscara. A máscara é definida por $m = 2^b - 1$, em que b indica o número de *bits* usados na parte baixa. Essas partes são combinadas por meio da operação XOR e somadas a uma constante ímpar, introduzindo não linearidade no processo. O valor resultante é então rotacionado à esquerda por ρ *bits* e combinado novamente com a palavra anterior por meio de XOR, promovendo a difusão dos *bits*. O programa armazena os valores gerados em uma lista, normalizando-os no intervalo $[0, 1)$, e monitora a ocorrência de ciclos ao registrar os estados anteriores.

Esse tipo de abordagem é interessante do ponto de vista conceitual, pois ilustra o uso de operações *bitwise* na construção de geradores pseudoaleatórios. Ideias semelhantes aparecem em geradores mais sofisticados, como aqueles baseados em operações de deslocamento e XOR (*Xorshift*), nos quais os parâmetros são cuidadosamente escolhidos para garantir melhores propriedades estatísticas.

2.4 Método de Fibonacci

O método de Fibonacci para geração de números pseudoaleatórios generaliza a sequência de Fibonacci clássica por meio de uma relação de recorrência que combina termos

anteriores da sequência. A forma padrão do método é definida por:

$$s_i = (s_{i-p} \star s_{i-q}) \bmod \alpha$$

onde $\star$ representa uma operação binária (tipicamente adição, subtração ou XOR), com $p > q$, e os parâmetros α, p, q sendo inteiros. A sequência deve ser inicializada com p valores conhecidos.

Considere um exemplo com:

- $\alpha = 16$
- $p = 5, q = 3$
- $s_0 = 1, s_1 = 1, s_2 = 2, s_3 = 3, s_4 = 5$.

Aplicando a fórmula $s_i = (s_{i-5} + s_{i-3}) \bmod 16$, obtém-se:

i	$s_{i-5} + s_{i-3}$	s_i	$r_i = \frac{s_i}{\alpha}$
5	$1 + 2 = 3$	3	0,1875
6	$1 + 3 = 4$	4	0,2500
7	$2 + 5 = 7$	7	0,4375
8	$3 + 3 = 6$	6	0,3750
9	$5 + 4 = 9$	9	0,5625
10	$3 + 7 = 10$	10	0,6250

Esse tipo de gerador é, em geral, mais robusto que os anteriores e pode apresentar períodos longos quando os parâmetros são bem escolhidos.

O código correspondente em Python é apresentado a seguir:

```
def fibonacci_aleatorio(alpha, p, q, semente, n):
    s = list(semente)
    resultados = []

    for i in range(n):
        novo = (s[-p] + s[-q]) % alpha
        ri = novo / alpha
        resultados.append((s[-p], s[-q], novo, ri))
        s.append(novo)

    print(f"{'s_{i-p}':>5} + {'s_{i-q}':>5} = {'s_i':>4} | {'r_i':>6}")
    print("-" * 32)
    for a, b, si, ri in resultados:
        print(f"{a:>5} + {b:>5} = {si:>4} | {ri:>6.4f}")
```

```
# Exemplo de uso
```

```
fibonacci_aleatorio(alpha=16, p=5, q=3, semente=[1, 1, 2, 3, 5], n=10)
```

3 Validação de geradores

Para verificar se uma sequência produzida por um gerador apresenta o comportamento esperado de uma sequência aleatória, utilizam-se testes estatísticos. Esses testes avaliam se os números gerados seguem a distribuição teórica desejada — geralmente, a distribuição uniforme no intervalo $[0, 1)$ — ou se há desvios significativos que indiquem padrões ou falhas no processo de geração.

Dentre os testes mais utilizados, destacam-se:

- **Teste do qui-quadrado (χ^2):** compara a frequência observada de valores em subintervalos com a frequência esperada de uma distribuição uniforme. Diferenças significativas podem indicar não aleatoriedade.
- **Teste de Kolmogorov-Smirnov:** compara a função de distribuição acumulada empírica dos dados com a função de distribuição acumulada teórica. É especialmente sensível a discrepâncias sutis, sendo mais indicado para amostras pequenas.

Esses testes são baseados na formulação de duas hipóteses complementares:

- **Hipótese nula (H_0):** assume que a sequência de números gerados segue a distribuição esperada, por exemplo, uma distribuição uniforme no intervalo $[0, 1)$. Em outras palavras, quaisquer diferenças entre as frequências observadas e esperadas são pequenas e atribuídas ao acaso.
- **Hipótese alternativa (H_1):** assume que a sequência de números gerados *não* segue a distribuição esperada. A discrepância observada é considerada grande demais para ter ocorrido apenas por variação aleatória.

O objetivo do teste é avaliar se há evidências suficientes nos dados para rejeitar H_0 . Para isso, calcula-se uma estatística de teste e a compara com um valor crítico, obtido da distribuição teórica apropriada, como a distribuição χ^2 . Se a estatística exceder o valor crítico para um nível de significância pré-definido, geralmente 5%, rejeita-se a hipótese nula em favor da alternativa. Caso contrário, conclui-se que não há evidência suficiente para rejeitar H_0 .

É importante destacar que não rejeitar H_0 não significa que a hipótese seja verdadeira, mas apenas que os dados são compatíveis com a distribuição assumida.

3.1 Teste do qui-quadrado

O teste do qui-quadrado de aderência verifica se uma sequência de números segue uma distribuição teórica esperada. Para isso, divide-se o intervalo $[0, 1)$ em k subintervalos de igual tamanho e conta-se quantos valores da amostra caem em cada subintervalo. A estatística do teste é dada por:

$$\chi^2 = \sum_{i=1}^k \frac{(f_i - E)^2}{E}$$

onde f_i é a frequência observada no i -ésimo subintervalo, e $E = \frac{n}{k}$ é a frequência esperada em cada classe sob a hipótese nula de distribuição uniforme, com n sendo o total de números gerados.

Por exemplo, considere a seguinte sequência de 18 números pseudoaleatórios no intervalo $[0, 1)$:

0,738	0,554	0,319	0,397	0,211	0,501
0,994	0,218	0,768	0,470	0,930	0,273
0,027	0,662	0,529	0,331	0,632	0,276

Dividindo-se o intervalo $[0, 1)$ em três partes iguais: $[0, \frac{1}{3})$, $[\frac{1}{3}, \frac{2}{3})$ e $[\frac{2}{3}, 1)$, são obtidas as seguintes frequências:

Observadas: $f = (7, 7, 4)$ Esperadas: $E = 6$

Com isso, é possível calcular a estatística do teste:

$$\chi^2 = \frac{(7-6)^2}{6} + \frac{(7-6)^2}{6} + \frac{(4-6)^2}{6} = \frac{1}{6} + \frac{1}{6} + \frac{4}{6} = 1,0$$

Como $k = 3$, o número de graus de liberdade do teste é $k - 1 = 2$. Esse valor representa quantas das frequências observadas podem variar livremente antes que as demais sejam determinadas por restrições do problema, por exemplo, a soma total das frequências.

No teste do qui-quadrado, o valor calculado da estatística é comparado com um valor crítico da distribuição χ^2 com o mesmo número de graus de liberdade, obtido a partir de tabelas estatísticas ou softwares. Esse valor crítico define o limite além do qual a diferença entre os dados observados e os esperados é considerada estatisticamente significativa.

Para um nível de significância de 5%, isto é, um risco de 5% de rejeitar incorretamente a hipótese nula, o valor crítico da distribuição χ^2 com 2 graus de liberdade é aproximadamente 5,991. Isso significa que, assumindo a hipótese nula verdadeira, de que os dados seguem uma distribuição uniforme, há apenas 5% de chance de a estatística χ^2 ultrapassar 5,991 apenas devido à variabilidade natural da amostragem.

Como o valor obtido foi $\chi^2 = 1,0$, bem abaixo do valor crítico, conclui-se que a diferença

entre as frequências observadas e esperadas é pequena o suficiente para ser atribuída ao acaso. Portanto, não há evidência suficiente para rejeitar a hipótese nula, e os dados são compatíveis com uma distribuição uniforme no intervalo considerado.

A **distribuição qui-quadrado** é uma distribuição de probabilidade contínua que surge naturalmente ao se somarem os quadrados de variáveis aleatórias independentes com distribuição normal padrão, isto é, com média zero e variância um. Ela é amplamente utilizada em testes estatísticos, como os de aderência e independência.

Propriedades relevantes da distribuição χ^2 :

- É assimétrica à direita, especialmente para poucos graus de liberdade.
- Os valores possíveis da estatística χ^2 são sempre não negativos.
- À medida que os graus de liberdade aumentam, a distribuição se aproxima de uma normal.

A intuição por trás do teste do qui-quadrado está relacionada ao fato de que, sob a hipótese nula, as diferenças entre frequências observadas e esperadas devem ser pequenas e atribuíveis à variabilidade natural dos dados. Quando essas diferenças são padronizadas e elevadas ao quadrado, elas formam uma medida global do desvio entre os valores observados e os esperados. Assim, valores muito altos da estatística indicam que os desvios são improváveis de terem ocorrido apenas ao acaso, sugerindo que a sequência de números não é compatível com a distribuição esperada.

A seguir, apresenta-se um programa em Python que aplica o teste do qui-quadrado a uma sequência de números gerados aleatoriamente. Os parâmetros n e k podem ser ajustados conforme a necessidade da análise.

```
import random
from scipy.stats import chi2

# Parametros do teste
n = 1000          # numero de valores gerados
k = 10            # numero de subintervalos
E = n / k         # frequencia esperada
alpha = 0.05      # nivel de significancia

# Geracao dos numeros
valores = [random.random() for _ in range(n)]

# Contagem das frequencias observadas
frequencias = [0] * k

# Divide o intervalo [0,1) em k classes
for v in valores:
```

```

    indice = int(v * k)
    frequencias[indice] += 1

# Calculo do qui-quadrado
chi2_calculado = sum((f - E) ** 2 / E for f in frequencias)

# Valor critico da distribuicao qui-quadrado
graus_de_liberdade = k - 1
chi2_critico = chi2.ppf(1 - alpha, graus_de_liberdade)

# Calculo do p-valor
p_value = 1 - chi2.cdf(chi2_calculado, graus_de_liberdade)

# Exibicao dos resultados
print(f"Frequencias observadas: {frequencias}")
print(f"Frequencia esperada em cada classe: {E:.1f}")
print(f"Estatistica qui-quadrado calculada: {chi2_calculado:.4f}")
print(f"Valor critico (alpha={alpha}): {chi2_critico:.4f}")
print(f"p-valor: {p_value:.4f}")

if chi2_calculado <= chi2_critico:
    print("\nResultado: Nao se rejeita H0 (os dados sao compatíveis com uniformidade)")
else:
    print("\nResultado: Rejeita-se H0 (os dados nao sao compatíveis com uniformidade)")

```

O programa utiliza `random.random()` para gerar números pseudoaleatórios uniformes em $[0, 1)$ e os distribui em k subintervalos, contando as frequências observadas. Em seguida, calcula a estatística qui-quadrado (`chi2_calculado`) para comparar essas frequências com as esperadas. As funções `chi2.ppf` e `chi2.cdf` do módulo `scipy.stats` são essenciais: `chi2.ppf(1-alpha, graus_de_liberdade)` retorna o valor crítico da distribuição qui-quadrado para o nível de significância escolhido, enquanto `chi2.cdf(chi2_calculado, graus_de_liberdade)` calcula a probabilidade acumulada até o valor da estatística, permitindo derivar o p-valor ($1 - \text{cdf}$). Com esses elementos, o programa decide se os dados são compatíveis com a distribuição uniforme, comparando a estatística calculada com o valor crítico ou analisando o p-valor.

3.2 Teste do qui-quadrado para a distribuição exponencial

O teste do qui-quadrado também pode ser utilizado para verificar a aderência dos dados a outras distribuições teóricas, como a distribuição exponencial. Para isso, é necessário conhecer a função de distribuição acumulada (FDA) da distribuição em questão e determinar as probabilidades teóricas associadas a cada classe de valores observados.

Considere, por exemplo, uma amostra de 1000 números gerados a partir de uma distribuição exponencial com parâmetro $\lambda = 1$, isto é, com densidade de probabilidade:

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

e divisão do intervalo $[0, \infty)$ em 5 classes com limites definidos de forma que cada uma tenha, aproximadamente, 20% da probabilidade total. Isto é, $P(X \leq x_i) = i \cdot 0,2$ para $i = 1, 2, \dots, 5$. Os limites dessas classes podem ser obtidos invertendo a FDA da exponencial, que é:

$$F(x) = 1 - e^{-\lambda x} \Rightarrow x_i = -\frac{1}{\lambda} \ln(1 - p_i)$$

onde $p_i = i \cdot 0,2$.

Para $\lambda = 1$, tem-se:

Classe 1: $[0, -\ln(0,8)) \approx [0, 0,223)$

Classe 2: $[0,223, -\ln(0,6)) \approx [0,223, 0,511)$

Classe 3: $[0,511, -\ln(0,4)) \approx [0,511, 0,916)$

Classe 4: $[0,916, -\ln(0,2)) \approx [0,916, 1,609)$

Classe 5: $[1,609, \infty)$

Geramos 1000 valores pseudoaleatórios com distribuição exponencial e contamos quantos caem em cada classe. Suponhamos que as frequências observadas foram:

Observadas: $f = (202, 208, 206, 196, 188)$, Esperadas: $E = 200$

Calculamos a estatística χ^2 :

$$\chi^2 = \sum_{i=1}^5 \frac{(f_i - E)^2}{E} = \frac{(202 - 200)^2}{200} + \frac{(208 - 200)^2}{200} + \frac{(206 - 200)^2}{200} + \frac{(196 - 200)^2}{200} + \frac{(188 - 200)^2}{200}$$

$$\chi^2 = \frac{4}{200} + \frac{64}{200} + \frac{36}{200} + \frac{16}{200} + \frac{144}{200} = 1,32$$

Com $k = 5$ classes e nenhum parâmetro estimado, supondo λ conhecido, os graus de liberdade são $k - 1 = 4$. O valor crítico para 5% de significância é aproximadamente 9,488. Como $1,32 < 9,488$, não se rejeita a hipótese nula: a sequência é compatível com uma distribuição exponencial de parâmetro $\lambda = 1$.

Este exemplo mostra que o teste do qui-quadrado pode ser adaptado para verificar a aderência a diferentes distribuições teóricas, desde que as frequências esperadas sejam corretamente determinadas com base na função de distribuição acumulada correspondente.

4 Geração de Variáveis Aleatórias

A maioria dos geradores de números pseudoaleatórios produz valores com distribuição uniforme no intervalo $[0, 1)$. No entanto, em aplicações práticas, é comum a necessidade de gerar variáveis com outras distribuições de probabilidade, como a exponencial, normal, binomial, entre outras.

Uma das técnicas mais utilizadas para essa transformação é o **método da função inversa de distribuição acumulada**, também conhecido como **método da inversa**. O procedimento baseia-se na seguinte ideia: a função de distribuição acumulada (FDA) de uma variável aleatória X associa a cada valor x a probabilidade de que $X \leq x$. O método da inversa consiste em inverter esse mapeamento: a partir de um valor u (sorteado de forma uniforme em $[0, 1)$), encontra-se o valor correspondente de x tal que $F(x) = u$.

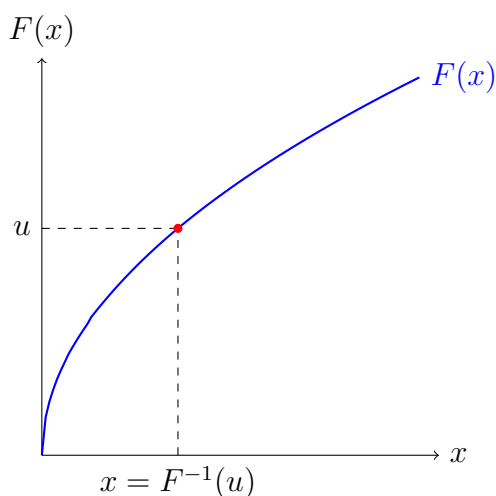
- Seja U uma variável aleatória uniforme em $[0, 1)$.
- Seja F a função de distribuição acumulada (FDA) da variável aleatória X que se deseja gerar.
- Então, define-se:

$$X = F^{-1}(U),$$

isto é, o valor de x cuja probabilidade acumulada é igual a U . Com isso, a variável X gerada possui a distribuição especificada por F .

Esse procedimento garante que a variável X gerada segue a distribuição desejada.

A figura a seguir ilustra o funcionamento do método da inversa. Sorteia-se um valor $u \in [0, 1)$, e localiza-se o valor x tal que $F(x) = u$. Esse valor x é a amostra gerada da variável aleatória X com distribuição acumulada F .



Considere a seguinte variável aleatória X , que pode assumir os valores 1, 2 e 3 com as seguintes probabilidades:

$$P(X = 1) = 0,2, \quad P(X = 2) = 0,5, \quad P(X = 3) = 0,3$$

A função de distribuição acumulada (FDA) associada é:

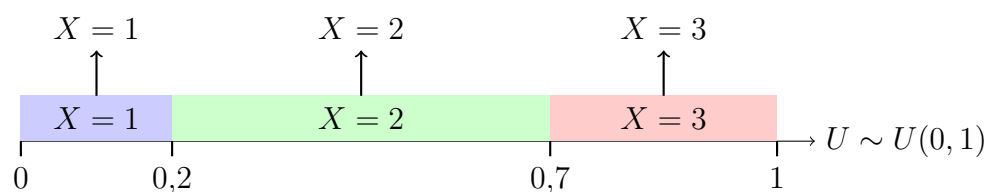
$$F_X(x) = \begin{cases} 0 & \text{se } x < 1 \\ 0,2, & 1 \leq x < 2 \\ 0,7, & 2 \leq x < 3 \\ 1, & x \geq 3 \end{cases}$$

Para gerar valores dessa variável discreta usando uma variável uniforme $U \sim U(0, 1)$, aplica-se o **método da inversão**. A ideia é a seguinte: sorteia-se um número aleatório $U \in [0, 1)$ e se determina o valor de X que corresponde ao menor x tal que $F(x) \geq U$.

Explicando passo a passo:

- Se $U < 0,2$, então o valor sorteado está dentro da probabilidade acumulada até o primeiro valor. Portanto, define-se $X = 1$.
- Se $0,2 \leq U < 0,7$, então U está entre os valores acumulados de $X = 1$ e $X = 2$. Define-se $X = 2$.
- Se $0,7 \leq U < 1$, então U está acima da probabilidade acumulada até $X = 2$ e abaixo de 1. Define-se $X = 3$.

A figura a seguir implementa essa correspondência entre a variável U e os valores de X :



O código Python a seguir gera 10 valores desta distribuição:

```
import random

def gerar_discreta():
    u = random.random()
    if u < 0.2:
        return 1
    elif u < 0.7:
        return 2
```



```

else:
    return 3

# Geração de 10 valores
amostras = [gerar_discreta() for _ in range(10)]
print("Amostras da distribuição discreta:", amostras)

```

Esse método garante que a variável X gerada siga exatamente a distribuição de probabilidade especificada no início da seção.

Para exemplificar uma distribuição contínua, considere a distribuição exponencial com parâmetro $\lambda > 0$, cuja função de distribuição acumulada é:

$$F(x) = 1 - e^{-\lambda x}, \quad x \geq 0$$

Inverter essa função resulta em:

$$F^{-1}(u) = -\frac{1}{\lambda} \ln(1 - u)$$

Como U é uniforme em $[0, 1)$, $1 - U$ também é uniforme, então costuma-se usar diretamente:

$$X = -\frac{1}{\lambda} \ln(U)$$

O código Python a seguir gera 10 valores respeitando a distribuição exponencial:

```

import random
import math

lambda_ = 1.5

# Geração de 10 valores exponenciais
exponenciais = [-math.log(random.random()) / lambda_ for _ in range(10)]

print("Amostras da distribuição exponencial:")
for x in exponenciais:
    print(f"{x:.4f}")

```

Um último exemplo considera a distribuição de Poisson. A distribuição de Poisson não possui uma função inversa analítica simples. Uma técnica clássica para gerar variáveis aleatórias de Poisson é o **algoritmo de Knuth**, que explora a relação entre a distribuição de Poisson e somas de variáveis exponenciais. O método pode ser descrito da seguinte forma:

- Fixe o parâmetro $\lambda > 0$.
- Calcule $L = e^{-\lambda}$.

- Inicialize $k = 0$ e $p = 1$.
- Enquanto $p > L$, faça:
 - Incrementar k .
 - Gerar $u \sim \mathcal{U}[0, 1)$ e atualizar $p \leftarrow p \cdot u$.
- Quando $p \leq L$, retornar $k - 1$ como o valor da variável de Poisson.

A implementação em Python é dada por:

```
import random
import math

def gerar_poisson(lamb):
    L = math.exp(-lamb)
    k = 0
    p = 1.0
    while p > L:
        k += 1
        p *= random.random()
    return k - 1

# Geração de 10 valores de Poisson com lambda = 3
poissons = [gerar_poisson(3.0) for _ in range(10)]

print("Amostras da distribuição de Poisson:", poissons)
```

A transformação de variáveis uniformes em outras distribuições é um passo fundamental em simulações. Em muitos casos, bibliotecas como `numpy.random` ou `scipy.stats` oferecem funções eficientes e precisas para gerar essas variáveis diretamente. No entanto, entender os métodos básicos – como o da inversa – é essencial para compreender e validar os resultados de simulações.

5 Considerações Finais

A geração de números aleatórios com diferentes distribuições de probabilidade constitui uma etapa essencial em diversas aplicações de simulação, amostragem e modelagem estatística. Nesta nota de aula, foram apresentados os fundamentos da geração de números pseudoaleatórios, com destaque para os principais algoritmos utilizados na construção de sequências com comportamento semelhante ao de variáveis uniformemente distribuídas no intervalo $[0, 1)$.

A partir da distribuição uniforme, é possível obter amostras de outras distribuições por meio de técnicas de transformação. O método da função inversa, embora limitado

a distribuições cuja função acumulada inversa possa ser determinada explicitamente ou aproximada numericamente, fornece uma abordagem simples e intuitiva, especialmente útil em contextos didáticos ou controlados. Distribuições discretas também podem ser simuladas com base em suas funções de distribuição acumulada.

Por fim, destaca-se que, embora bibliotecas computacionais ofereçam funções prontas para a geração de variáveis aleatórias com diferentes distribuições, a compreensão dos princípios teóricos que fundamentam esses métodos é fundamental para a validação dos resultados e para o desenvolvimento de simulações confiáveis e adequadas a diferentes contextos de aplicação.

Referências

ALENCAR, M. S. de. *Probabilidade e processos estocásticos*. 1a. ed. São Paulo: Érica, 2008.

CHWIF, L.; MEDINA, A. C. *Modelagem e simulação de eventos discretos: teoria & prática*. 2a. ed. São Paulo: Ed. dos autores, 2006.

FILHO, C. P. *Introdução à simulação de sistemas*. 1a. ed. Campinas: Editora da Unicamp, 1995.

HUNT, J. *A Beginners Guide to Python3 Programming*. 1a. ed. Cham: Springer, 2020.

MARCONTES, G. A. B. *Matemática com Python: um guia prático*. 1a. ed. São Paulo: Novatec, 2018.

MENEZES, N. N. C. *Introdução à Programação com Python: algoritmos e lógica de programação para iniciantes*. 1a. ed. São Paulo: Novatec, 2010.